UNIVERSITY OF NORTH CAROLINA AT GREENSBORO

DEPARTMENT OF COMPUTER SCIENCE

CSC362: System Programming

Assignment 3

Question 1: Dynamic Linking, Shared Libraries, and Pushing Critical Updates for Security

You are developing an application, **secure_transaction_app**, which handles sensitive transaction operations and logging. However, there are critical issues in the current version of the app, such as memory leaks, dangling pointers, and a null byte vulnerability. You will address each issue by pushing updates to the respective libraries while dynamically linking them to the main application.

The following files are provided to you:

1. **main.c**: The main program that uses both a transaction library and a logging library.
2. **translib_v1.c**: The first version of the transaction library, which contains a memory leak issue.
3. **loglib_v1.c**: The first version of the logging library, which contains a dangling pointer issue.
4. **translib_v2.c**: The second version of the transaction library, which fixes the memory leak.
5. **loglib_v2.c**: The second version of the logging library, which fixes the dangling pointer issue.
6. **translib_v3.c**: The third version of the transaction library, which fixes a null byte issue.

Your task is to compile these files into shared libraries, dynamically link them to the main program, and push updates to address each vulnerability.

Tasks:

1. **Compile and link libraries**:
   - Compile **translib_v1.c** into a shared library `libtrans.so.1.0`.
   - Compile **loglib_v1.c** into a shared library `liblog.so.1.0`.
   - Compile **main.c** and dynamically link it to both libraries.
   - Ensure that **secure_transaction_app** runs with both **libtrans.so.1.0** and **liblog.so.1.0**.

2. **Push updates**:
   - **Update 1**: Compile **translib_v2.c** into `libtrans.so.2.0`, fixing the memory leak.
   - **Update 2**: Compile **loglib_v2.c** into `liblog.so.2.0`, fixing the dangling pointer issue.
   - **Update 3**: Compile **translib_v3.c** into `libtrans.so.3.0`, fixing the null byte issue.
   - Update **secure_transaction_app** to use the new versions of the libraries after each update without recompiling **main.c**.
   - Use symbolic links to switch between versions.

3. **Write All Commands**:
   - As part of your answer, list all the `gcc` and linking commands you used to compile the libraries and push the updates. Ensure that each command leads to the corresponding expected output.

Expected Output:

- **Before any update**:

```
Transaction Error: Memory leak detected during
transaction!
LOG: Transaction started. Logging information...
Logging Error: Dangling pointer detected in log!
Processing transaction...
LOG: Transaction completed successfully.
```

- **After Update 1 (memory leak fix)**:

```
Transaction in progress...
Memory management secure. Remaining issues: Dangling
pointer, Null byte.
LOG: Transaction started. Logging information...
Logging Error: Dangling pointer detected in log!
Processing transaction...
LOG: Transaction completed successfully.
```

- **After Update 2 (dangling pointer fix)**:

```
Transaction in progress...
Memory management secure. Remaining issue: Null byte.
LOG: Transaction started. Logging information...
Logging secure. Remaining issue: Null byte.
Processing transaction...
LOG: Transaction completed successfully.
```

- **After Update 3 (null byte fix)**:

```
Transaction in progress...
Null byte issue resolved. No remaining issues.
Transaction is secure.
LOG: Transaction started. Logging information...
Logging secure. No remaining issues.
Processing transaction...
LOG: Transaction completed successfully.
```